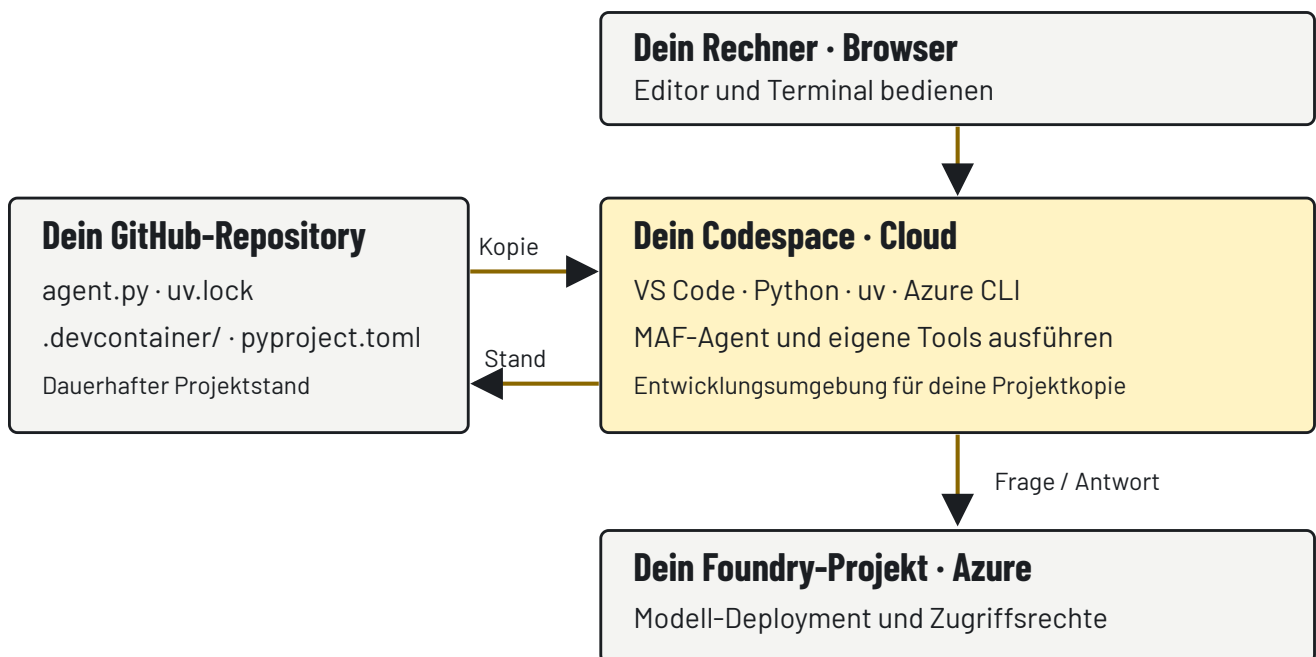


DEIN EIGENES MAF-PROJEKT

Unsere Vorlage übernehmen und im Browser selbstständig weiterentwickeln.

01 DIE UMGEBUNG KANNST DU MITNEHMEN

Für ein neues Projekt nutzt du den eigenständigen MAF-Starter. Er enthält den kleinen Beispielagenten und die Umgebungsconfiguration. Du brauchst ein GitHub-Konto und einen Browser. Aus der [Pondhouse-MAF-Starter](#) erstellst du dein eigenes Projekt. Der Codespace richtet Python 3.13, uv, Azure CLI und die festgelegten MAF-Pakete automatisch ein. Dein Agent läuft dort und ruft das Modell in Foundry auf.



02 DAS ERGEBNIS

- Eine eigene Projektkopie, deren Code du ändern und aufbewahren kannst.
- Eine reproduzierbare Cloudumgebung mit denselben Paketversionen.
- Ein kleiner lauffähiger Agent in agent.py.

GitHub organisiert Code und Umgebung. Azure stellt den Modellzugang bereit. Beide Konten haben ihre eigenen Berechtigungen und Kosten. Für ein Firmenprojekt verwendest du den von deiner IT freigegebenen GitHub- und Azure-Zugang.



TRANSFER · VORLAGE UND UMGEBUNG

DEIN PROJEKT ANLEGEN

03 EINE EIGENE KOPIE ERSTELLEN

- 1 Auf GitHub die [Vorlage pondhouse-data/maf-starter](#) öffnen. **Use this template** → **Create a new repository** wählen.
- 2 Unter **Owner** dein Konto oder deine freigegebene Organisation auswählen. Als Namen beispielsweise `mein-maf-agent` eintragen und die Sichtbarkeit passend zum Projekt wählen. **Create repository from template** anklicken.
- 3 Im neuen Repository **Code** → **Codespaces** öffnen und den Codespace auf `main` erstellen. GitHub zeigt den Account für die Abrechnung. Auf den Editor und das Ende der automatischen Einrichtung warten.
- 4 **Terminal** → **New Terminal** öffnen und im Hauptordner ausführen:

```
uv run python -c "from agent_framework import Agent; from agent_framework.foundry import FoundryChatClient; print('MAF bereit')"
```

Erwartet: MAF bereit. Das prüft den Import der installierten MAF-Pakete. Den Modellzugang prüfst du auf der nächsten Seite mit einem echten Aufruf.

04 WELCHE DATEI MACHT WAS?

DATEI / ORDNER	AUFGABE
<code>.devcontainer/devcontainer.json</code>	Beschreibt Werkzeuge, Editor-Erweiterungen und den Einrichtungsbefehl des Codespace.
<code>.devcontainer/Dockerfile</code>	Legt die Python-Basis und die uv-Version fest.
<code>pyproject.toml</code> / <code>uv.lock</code>	Definieren die Python-Version, benötigte Pakete und ihre exakten Versionen.
<code>.env.example</code> / <code>.env</code>	Vorlage und deine eigene Konfiguration für Projekt-Endpoint und Modell-Deployment.
<code>agent.py</code>	Vollständiges Beispiel für deinen eigenen Agenten; Code auf Seite 4.
<code>.gitignore</code>	Lässt die private <code>.env</code> , installierte Pakete und Python-Cache beim Speichern ins Repository aus.

`uv sync --frozen` baut die Paketumgebung anhand von `uv.lock` auf. `uv run` startet ein Programm darin. Die Konfiguration im Projekt macht die Umgebung wieder aufbaubar.

TRANSFER · EIGENER MODELLZUGANG

DEINEN AGENTEN VERBINDEN

05 DEIN FOUNDRY-MODELL BEREITSTELLEN

Für eigene Projekte brauchst du ein erreichbares Foundry-Projekt, ein bereitgestelltes Chat-Modell und die Berechtigung deines Azure-Kontos zum Modellaufruf. Nutze dafür ein vorhandenes Firmenprojekt oder lasse den Zugang mit deiner IT einrichten. Bei einem Firmenprojekt kann die IT beispielsweise die passende Foundry-User-Rolle zuweisen.

Öffne dein Projekt unter ai.azure.com. Übernimm den **Projekt-Endpunkt** aus der Projektübersicht und den **Deploymentnamen** aus der Modellbereitstellung. Erstelle im Codespace zuerst deine Konfigurationsdatei und trage danach deine eigenen Werte im Editor ein.

```
cp .env.example .env
chmod 600 .env
```

Die Vorlage enthält die Namen der beiden benötigten Einstellungen.

EINTRAG IN .ENV	DEIN WERT
FOUNDRY_PROJECT_ENDPOINT	Der Projekt-Endpunkt aus Foundry, einschließlich /api/projects/...
FOUNDRY_MODEL	Der Name deines bereitgestellten Modells, so wie er im Projekt hinterlegt ist.

`agent.py` liest diese beiden Einträge. Für den ersten Modellaufruf reicht die Ausführung im Codespace. Wenn du später einen dauerhaft erreichbaren Dienst benötigst, knüpfe an den Hosting-Weg aus Ü13 an.

06 ANMELDEN UND AUSPROBIEREN

Im Codespace-Terminal mit dem Azure-Konto für dein eigenes Projekt anmelden. Wähle bei der Anmeldung den passenden Tenant und die Subscription.

```
az login --use-device-code
az account show --query "{Konto:user.name,Tenant:tenantId,Subscription:name}"
```

Bestätige den Gerätecode im eigenen Browserfenster. Die Kontoprüfung zeigt, wer im Codespace bei Azure angemeldet ist.

```
uv run python agent.py "Erkläre, wozu ein Agent Tools verwendet."
```

ERWARTET Eine kurze deutsche Antwort mit einem Beispiel. Der genaue Wortlaut kann variieren.

Öffne `agent.py`, ändere `instructions` auf „Antworte in genau einem Satz.“ Speichere die Datei und starte denselben Befehl erneut. Damit hast du eine eigene Agentenregel geändert und ihre Wirkung am Modell geprüft.

07 WENN DER AUFRUF SCHEITERT

Fehlende Werte: `.env` im Hauptordner prüfen. 401/403: Azure-Konto, Tenant und Berechtigung mit der IT klären.



TRANSFER · DER VOLLSTÄNDIGE BEISPIELAGENT

DIE ZEILEN VERSTEHEN

`agent.py` liegt im Hauptordner der Vorlage. Du kannst die Datei öffnen, anpassen und mit `uv run python agent.py` ausführen.

```
"""Ein eigener kleiner MAF-Agent: uv run python agent.py 'Deine Frage.'"""
import asyncio
import os
import sys

from agent_framework import Agent
from agent_framework.foundry import FoundryChatClient
from azure.identity.aio import AzureCliCredential
from dotenv import load_dotenv

async def main():
    load_dotenv()
    endpoint = os.getenv("FOUNDRY_PROJECT_ENDPOINT")
    model = os.getenv("FOUNDRY_MODEL")
    if not endpoint or not model:
        raise SystemExit("Trage FOUNDRY_PROJECT_ENDPOINT und FOUNDRY_MODEL in .env ein.")

    async with AzureCliCredential() as credential:
        client = FoundryChatClient(
            project_endpoint=endpoint, model=model, credential=credential
        )
        agent = Agent(
            client=client,
            name="MeinAssistent",
            instructions="Erkläre kurz auf Deutsch. Nenne ein konkretes Beispiel.",
        )
        question = " ".join(sys.argv[1:]) or "Wozu dient ein Tool bei einem Agenten?"
        response = await agent.run(question)
        print(response.text)

if __name__ == "__main__":
    asyncio.run(main())
```

08 WAS DU HIER ÜBERNIMMST

`load_dotenv()` liest deine Projektwerte. `AzureCliCredential` verwendet deine Azure-CLI-Anmeldung. `FoundryChatClient` verbindet sich mit deinem Modell. `Agent` bekommt Namen und Instructions; `agent.run` ruft ihn auf. Die Frage kommt aus dem Terminalargument. `asyncio.run` startet den Ablauf, `await` wartet auf die Modellantwort.

Erweitere diesen Agenten mit eigenen Tools nach dem Muster aus Ü9. Sessions, Middleware und Workflows folgen den Beispielen aus Ü10–Ü12. Behalte zunächst die getesteten Paketversionen; eine Aktualisierung ist eine eigene Projektänderung.



■ TRANSFER · STAND AUFBEWAHREN UND FORTSETZEN

DEIN PROJEKT BEHALTEN

09 CODE IM EIGENEN REPOSITORY SPEICHERN

Speichern im Editor bewahrt die Datei im Codespace auf. Um den Stand auch im Repository auf GitHub aufzubewahren:

- 1 Links im Editor **Source Control** öffnen. Prüfe die geänderten Dateien und wähle **+** neben deinem Code, um diese Änderungen zu übernehmen.
- 2 Eine Beschreibung wie „Eigene Agentenregel ergänzt“ eintragen. Im Menü am **Commit**-Button **Commit & Push** wählen. Commit hält einen Dateistand fest; Push überträgt ihn in dein eigenes Repository.
- 3 Das Repository auf GitHub öffnen und die geänderte Datei kontrollieren. Für den nächsten Tag denselben Codespace über github.com/codespaces wieder öffnen.

.gitignore lässt die private .env und installierte Pakete aus. Wenn du aus dem Repository einen neuen Codespace erstellst, wird die Umgebung neu eingerichtet. Erstelle dort erneut .env aus .env.example, trage deine eigenen Modellwerte ein und melde dich erneut an.

10 DEINEN TRAININGSSTAND SEPARAT AUFBEWAHREN

Für deine bearbeiteten Übungsdateien kannst du den bestehenden Trainings-Codespace zusätzlich in ein eigenes Repository überführen: **Source Control** öffnen, deine Änderungen mit **+** auswählen und mit **Commit** festhalten. Dann **Publish Branch** wählen, einen Projektnamen eingeben und die passende Sichtbarkeit auswählen. GitHub legt das Repository unter deinem Konto an. Dieser Weg bewahrt deine bearbeiteten Übungsdateien auf.

Für eine zusätzliche Dateikopie erstellt das mitgelieferte `sichern.py` ein ZIP: `uv run python sichern.py`, dann `mein-training.zip` im Explorer rechts anklicken → **Download**. Die Ausschlüsse stehen im Skript.

11 ZUGANG UND LAUFENDER BETRIEB

Stoppe deinen Codespace nach der Arbeit über ... → **Stop codespace**. Prüfe das Codespaces-Kontingent und den Account für die Abrechnung. Die Aufbewahrungsfrist des Codespace und der gespeicherte Projektstand im Repository sind getrennt. Für spätere Modellaufrufe brauchst du weiterhin einen gültigen Azure-Zugang; vereinbare für eigene Projekte einen eigenen Modellzugang. Copilot-Unterstützung hat eine separate Berechtigung.

12 WEITERLESEN

- [GitHub: eigenes Repository aus einer Vorlage](#)
- [GitHub: Trainings-Codespace in ein eigenes Repository übernehmen](#)
- [GitHub: Änderungen speichern und übertragen](#)
- [GitHub: Aufbau einer Dev-Container-Konfiguration](#)
- [uv: Projektdateien und Paketumgebung](#) · [Microsoft: MAF mit Foundry](#)
- [GitHub: Codespaces-Kontingente und Abrechnung](#)

Quellen abgerufen 06.10.2026. Beispielcode für die Versionen der Trainingsvorlage. Künz-Beispieldaten sind synthetisch.